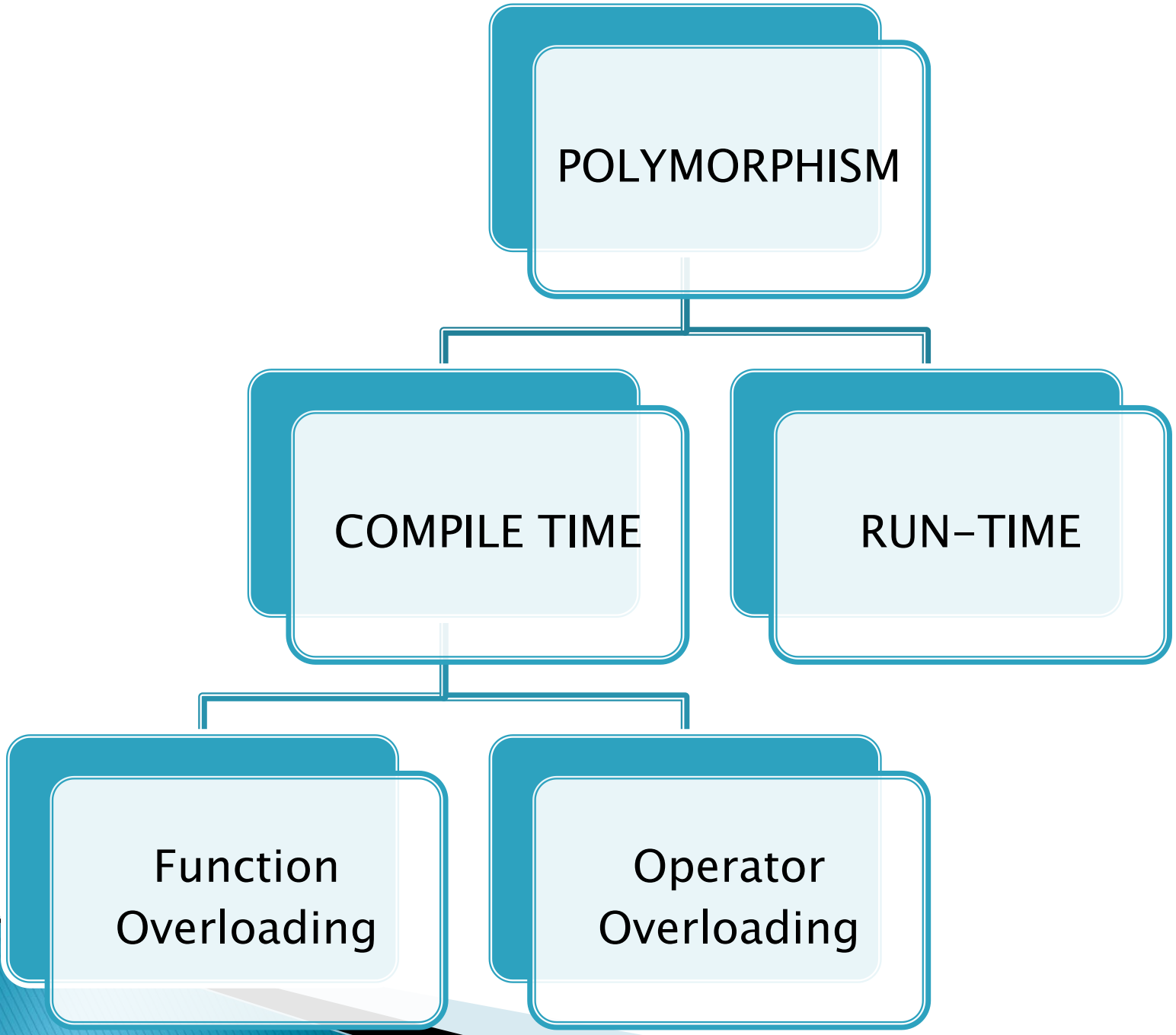
Run-time Polymorphism by Virtual Functions


POLYMORPHISM
COMPILE TIME
RUN-TIME
Function Overloading
Operator Overloading

POINTER TO OBJECT

- Pointer to object is a variable containing an address of an object.
- It is similar to a pointer to any other variable. The normal address-of operator has to be used to get the address of an object.

```
class Demo
{
// Body of the class
}
Demo DemoObj;
Demo *PtrDemoObj;
PtrDemoObj = &DemoObj;
```

this POINTER

- The this pointer is a pointer to an object invoked by the member function.
- Suppose one writes DemoObj.DispDemo(), then a function call to this pointer will return the address of DemoObj.
- In other words, this pointer is a pointer to DemoObj.

```
person Elder(person
OtherPerson)
{
      if(Age >
OtherPerson.Age)
       return *this;
      else
       return OtherPerson;
}

main()
{
      person p1,p2,p3;
      p1=p2.Elder(p3);
}
```

COMPATIBILITY OF DERIVED AND BASE CLASS POINTERS

```
BaseClass BC;
DerivedClass DC : public BaseClass;
BaseClass *PtrBC;
DerivedClass *PtrDC;
```

```
PtrBC = &BC; // Obvious, pointer and content are of similar types
PtrDC = &DC; // Here too
PtrBC = &DC; // This is done without any casting
// The following is not allowed
PtrDC = &BC
```

Subobject Concept

```
BaseClass1 BC1;
BaseClass2 BC2;
DerivedClass DC : public BaseClass1, BaseClass2;
BaseClass1 *PtrBC1;
BaseClass2 *PtrBC2;
DerivedClass *PtrDC;
```

Now, consider the following statements:

```
PtrBC1 = &DC;
PtrBC2 = &DC;
```

BASE CLASS AND DERIVED CLASS MEMBER FUNCTIONS

- It is possible for two different member functions, one defined in the base class and another defined in the derived class, to have the same name. The function in the derived class is known as an *overridden function*. The process is known as overriding the said function.

```
#include <iostream>
using namespace std;
class Shape
{
int LineStyle;
int FillColour;
public:
// Non-virtual function
void draw()
{
cout << "Shape is drawn \n";
}
};
class Circle : public Shape
{
int Radius;
int CentrePointX;
int CentrePointY;
public:
void draw()
{
cout << "Circle is drawn \n";
}
};
```

```
int main()
{
Shape SomeShape, *PtrShape;
Circle Ring;
SomeShape.draw();
Ring.draw();
PtrShape = &Ring;
PtrShape->draw();
/* This calls the draw function of the
base class and not the
derived class where the pointer is
pointing. */
}
```

Output

```
Shape is drawn
Circle is drawn
Shape is drawn
```

If Virtual Function in base class then

```
virtual void draw()
{
    cout << "Shape is drawn \n";
}
Shape is drawn
Circle is drawn
Circle is drawn
```

Virtual Destructors

```
class BaseClass
{
public:
BaseClass()
{
cout << "BaseClass constructor ..." << endl;
}
~BaseClass()
{
cout << "BaseClass destructor ..." << endl;
}
};

class DerivedClass : public BaseClass
{
public:
DerivedClass()
{
cout << "DerivedClass constructor ..." << endl;
}
~DerivedClass()
{
cout << "DerivedClass destructor ..." << endl;
}
};
```

```
void main()
{
BaseClass* ptrBase;
ptrBase = new DerivedClass();
delete ptrBase;
/* This should ideally call for
DerivedClass destructor, but will
call only BaseClass destructor,
because the destructor is not
virtual.*/
getchar();
}
```

Output

BaseClass constructor ...
DerivedClass constructor ...
BaseClass destructor ...

Cont…

```
virtual ~BaseClass()
{
cout << "BaseClass class destructor ..." << endl;
}
BaseClass constructor ...
DerivedClass constructor ...
DerivedClass destructor ...
BaseClass destructor ...
```

PURE VIRTUAL FUNCTIONS

```
class Shape
{
        public:
        virtual void draw()=0;
};
```

- Pure virtual functions do not have the body where they are defined, but they can still have it outside the class.
- Pure virtual functions of base class force derived class to implement body of pure virtual function.
- Class that has at least one pure virtual function is known as abstract class.